受付	月　日	評価	

情報知能・工学実験　報告書

実験題目	

実験実施日	令和 年 月 日
第 1 回	令和 年 月 日
第 2 回	令和 年 月 日
第 3 回	令和 年 月 日
第 4 回	令和 年 月 日
第 5 回	令和 年 月 日
第 6 回	令和 年 月 日

学籍番号	氏名

概要	

1　目的

近年のネットワーク技術の発展により Web サービスが日常において不可欠な技術となっている。そのため、Web サービスの停止そのものが社会に与える影響が大きい。Web サービスの停止は外部からの攻撃によって起こされることがある。その種類は様々であるが、一つに「DoS 攻撃」というものがある。DoS 攻撃はセキュリティの脆弱なコンピュータに対して使用される。DoS 攻撃をされたコンピュータはロードが急増し、サービスの提供が阻害されてしまうことが発生する。攻撃されたサービス事業者はこの攻撃によって得られる報酬が低減してしまう。よって Web サービスを運営するにあたって DoS 攻撃の対策は必要不可欠である。

本実験において、ネットワークの基本構造について知り、DoS 攻撃に対する対策法を身に着けることを目的とする。

2　解説

2.1　DoS 攻撃について

DoS(Denial-of-Service) 攻撃は特定のサーバやネットワークデバイスの web サービスを停止させる攻撃手段の一つである。
DoS 攻撃の一つに DDoS(distributed Denial-of-Service) 攻撃がある。DDoS 攻撃は、たくさんのコンピュータが同時に攻撃するプログラムを特定のサーバに対して実行する攻撃方法である。

Dos 攻撃の方法には、サーバの脆弱性を悪用する攻撃とサーバに過負荷をかける攻撃の 2 つがある。サーバに脆弱性がある場合は、攻撃者に技術的なアドバンテージを利用されることによって攻撃を仕掛けられる。ただし、脆弱性を悪用する攻撃はソフトウェアのアップデートによって防ぐことができる。サーバに過負荷をかける攻撃は、意図的に通信手段をブロックすることやサーバが処理しきられないほどのロードをすることによってネットワークを混雑させることができる。そのため、ネットワークとサーバの間を無駄なパケットで埋め尽くすので、対策をサーバ内で施すことは難しいとされている。一方で、サーバ側ができる対策としてはファイヤーウォールを設定するという方法がある。

いかに様々な Dos 攻撃について記述する。

2.1.1　UDP Flood

通信プロトコルの一つである UDP の特性を悪用する攻撃方法である。長時間に大量の UDP パケットを送信することで攻撃を実行する。

2.1.2　Ping Flood

通信情報を取得するプロトコル ICMP を悪用する攻撃方法である。大量の ICMP パケットを送り付けることによって攻撃する。

2.1.3　SYN Flood

TCP 通信を確立する SYN を大量に送りつける攻撃方法である。受信側の攻撃対象のサーバは SYN パケットを送り返した際に ACK をタイムアウトまで待ち続けなければならないため、メモリを消費してしまうとい

う攻撃方法である。

2.1.4 Connection Flood

コネクションが確立した状態を長時間つづけることでソケットを占有する攻撃方法である。

2.1.5 HTTP GET Flood

TCP コネクションが確立したのちに HTTP GET リクエストを大量に送信する攻撃方法である。シンプルな攻撃手段として F5 攻撃がある。F5 はブラウザ上で押下することで web ページをリロードすることが可能であり、HTTP GET を短時間で大量に送ることが可能である。

2.2 仮想マシンについて

仮想マシンはコンピュータの操作をエミュレートするソフトウェアであり、この技術を使うことで一つの物理コンピュータで複数の OS を扱うことが可能となる。例えば、Windows OS 上で Linux OS の環境を構築することが可能なる。web サービスはここ数年で大規模化され、それに伴いコストカットが重要となった。そこで、コストの軽減のために一つのコンピュータで複数の OS を扱えるような仮想マシンの技術が開発された。

仮想マシンを動かすためのソフトウェアを VMM(Virtual Machine Monitor0) と呼ぶ。VMM にはホスト型 VMM とハイパーバイザ型 VMM の 2 種類がある。ホスト型 VMM はホスト OS 上で実行され、ハイパーバイザ型 VMM はインストールと実行が直接ハードウェア上で実行される VMM である。ここで、ハードウェアに直接ダウンロードされた OS をホスト OS と呼ぶ。ホスト型 VMM はアプリケーションとして扱うことができるため、インストールや使い方が簡単である。一方、ハイパーバイザ型 VMM はハードウェアとの間のレイヤーがホスト型よりも少ないため、実行時のオーバヘッドが少なくなる。一般的な VMM として、VirtualBox や VMware はホスト型、KVM(Kernel-based Virtual Machine) はハイパーバイザ型である。本実験では、ハイパーバイザ型を使用する。

3 実験環境

本実験で使用した Web サーバのスペックを以下の表 1 に示す。
ここで、CPU のスペックの確認のコマンドは”lscpu”、メモリのスペックの確認のコマンドは”free -h”、ストレージのスペックの確認のコマンドは”lsblk”、ネットワークのスペックの確認のコマンドは”ip addr”、OS のスペックの確認のコマンドは”cat /etc/os-release”を使用した。

表 1: Web サーバのスペック

CPU	1 コア 3.4GHz
メモリ	500MB
ストレージ	7.6GB
IP アドレス	192.168.1.197
OS	Debian GNU/Linux 7

本実験で使用した攻撃用サーバのスペックを以下の表 2 に示す。

表 2: 攻撃用サーバのスペック

CPU	1 コア 3.4GHz
メモリ	500MB
ストレージ	8GB
IP アドレス	192.168.2.198
OS	Debian GNU/Linux 7

また、攻撃用サーバにおいては、一つのサーバ内に複数の IP アドレスを生成するためセカンダリ IP アドレスが、”192.198.197.1”から”192.198.197.150”まで生成されている。

実験に使用する攻撃ツールとして、”curl-loader”を使用する。curl-loader は C 言語で書かれた Web アプリケーションに対する攻撃ツールの一つである。一つのマシン上で複数の IP アドレスを使用して、リクエストの再生成を行う DDoS 攻撃の一つである。

4 実験課題

4.1 基礎実験

curl-loader の実行時にパラメータを変更し、CPU の使用率、メモリの使用率、アクセスログ、などを調査する。本実験では最初に環境を初期化するために、攻撃対象の web サーバを再起動する。その際のコマンドを以下のソースリスト 4.1 に示す。

ソースリスト 4.1：攻撃対象の web サーバの再起動コマンド

```
$ sudo /etc/init.d/apache2 restart
```

また、実験前にキャッシュをクリアするために、以下のソースリスト 4.1 のコマンドを実行する。

ソースリスト 4.1：キャッシュクリアコマンド

```
$ sudo sysctl -w vm.drop_caches=3
```

サーバの状態を観測するのに top コマンドと apache のアクセスログを使用する。top コマンドは以下のソースリスト 4.1 のように引数を設定することができる。

ソースリスト 4.1：top コマンドの引数設定

```
$ sudo top -b -d 1 -n 180 > top.log
```

このコマンドは、top コマンドをバッチモードで実行し、1 秒ごとに 180 回の更新を行い、その結果を top.log に出力するものである。こうすることで、サーバの状態を 1 秒おきに観測することができる。

top コマンドから CPU の使用室のみを抽出しファイルに書き出すコマンドを以下のソースリスト 4.1 に示す。

ソースリスト 4.1：CPU 使用率の抽出コマンド

```
$ grep "%Cpu" top.log | awk '{print $2}' > cpu.log
```

top コマンドからメモリの使用率のみを抽出しファイルに書き出すコマンドを以下のソースリスト 4.1 に示す。

ソースリスト 4.1：メモリ使用率の抽出コマンド

```
$ grep "Mem" top.log | awk '{print $5/$3}' > mem.log
```

access.log から 1 秒ごとのアクセス数を抽出してファイルに書き出すコマンドを以下のソースリスト 4.1 に示す。

ソースリスト 4.1：アクセス数の抽出コマンド

```
$ grep "192.168.197." access.log | awk '{print $4}' | sort | uniq -c > freq.log
```

本実験では、curl-loader の設定ファイルである sample.cnf の"TIMER_AFTER_URL_SLEEP"の値を 200, 400, 800 に変更してその結果を比較する。この時、"TIMER_AFTER_URL_SLEEP"はリクエストの完了後に次のリクエストをするまで curl-loader が待機する時間をミリセカンド単位で表すパラメータである。

curl-loader の実行コマンドは以下のようになる。

```
$ sudo curl-loader -f sample.cnf
```

curl-loader は root 権限を持ったうえで実行しなければならない。ただし、&によってコマンドをつなぐことは後続のコマンドも root 権限で実行されてしまうため、注意が必要である。

4.2 攻撃者判定実験

攻撃者を検知してフィルタリングするシェルスクリプトを web サーバ上で作成する。加えて、4.1 で行った実験をもう一度行い、フィルタリング設定時のふるまいを調査する。その際に以下の箇条書きの点を満たすように調査する。

- TIMER_AFTER_URL_SLEEP の値を 100, 200, 400 に変更した際の CPU 使用率、メモリ使用率、アクセスログを測定する。
- フィルタリングをかけた時と書けない時の値を測定する。
- iptable の DROP に基づいてフィルタリングを行う。
- フィルタリングする攻撃者の基準として、毎秒三回以上アクセスしている IP アドレスに絞るようにする。
- 攻撃者の見地は"/var/log/apache2/p3/access.log"のアクセスログをもとに判断する。

また、Web サーバのホームディレクトリ上に"attecker_search"というディレクトリを作成し、攻撃者の判定に使用したフィルを保存する。

5　実験結果

5.1　基礎実験

最初に、apache を再起動した際のコマンドの実行結果とキャッシュをクリアした際のコマンドの実行結果を以下のソースリスト 5.1 に示す。

ソースリスト 5.1：攻撃対象の web サーバの初期化コマンドの実行結果

```
$ sudo /etc/init.d/apache2 restart
[ ok ] Restarting apache2: apache2 ... waiting .
$ sudo sysctl -w vm.drop_caches=3
vm.drop_caches = 3
```

以下の図 1 に TIMER_AFTER_URL_SLEEP の値を 200, 400, 800 に変更した際の CPU 使用率のグラフを示す。

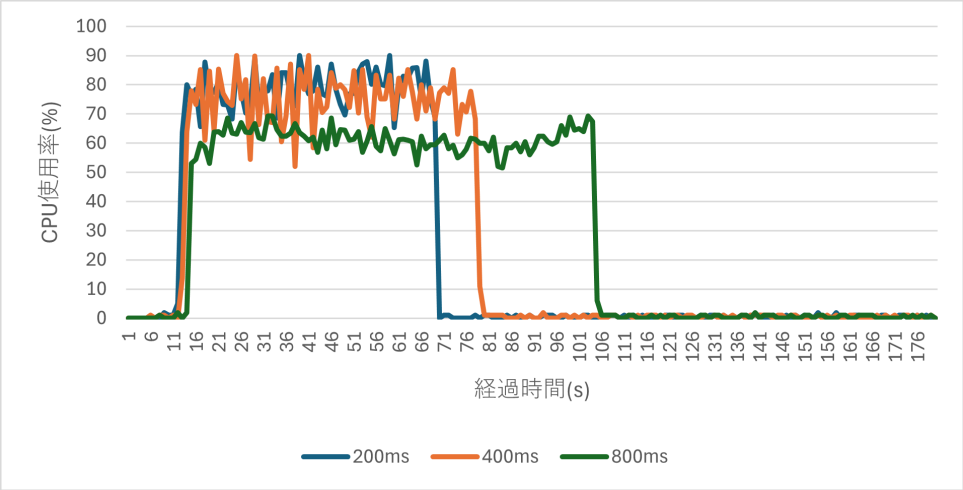

図 1: 基礎実験の CPU 使用率のグラフ

以下の図 2 に TIMER_AFTER_URL_SLEEP の値を 200, 400, 800 に変更した際のメモリ使用率のグラフを示す。

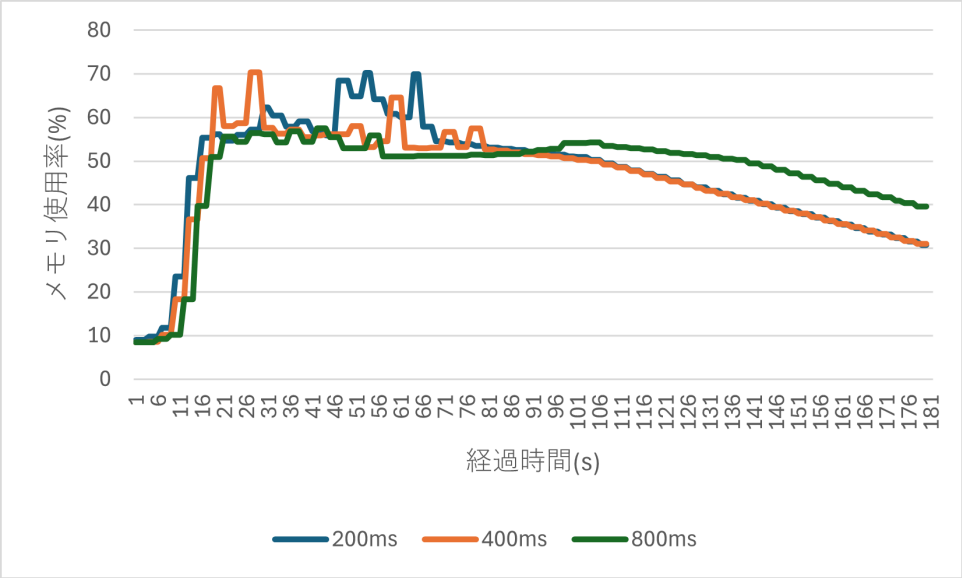

図 2: 基礎実験のメモリ使用率のグラフ

以下の図 3 に TIMER_AFTER_URL_SLEEP の値を 200, 400, 800 に変更した際のアクセス数のグラフを示す。

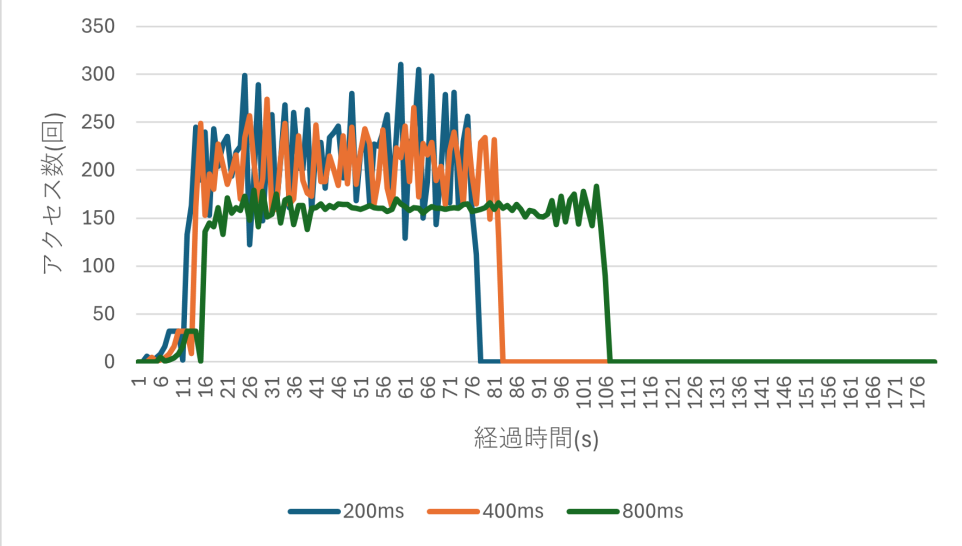

図 3: 基礎実験のアクセス数のグラフ

図 1 から、TIMER_AFTER_URL_SLEEP の値が 200 と 400 の時の CPU 使用率は、800 の時よりも高いことがわかる。また、TIMER_AFTER_URL_SLEEP のあたい

5.2　攻撃者判定実験

6　調査課題

参考文献